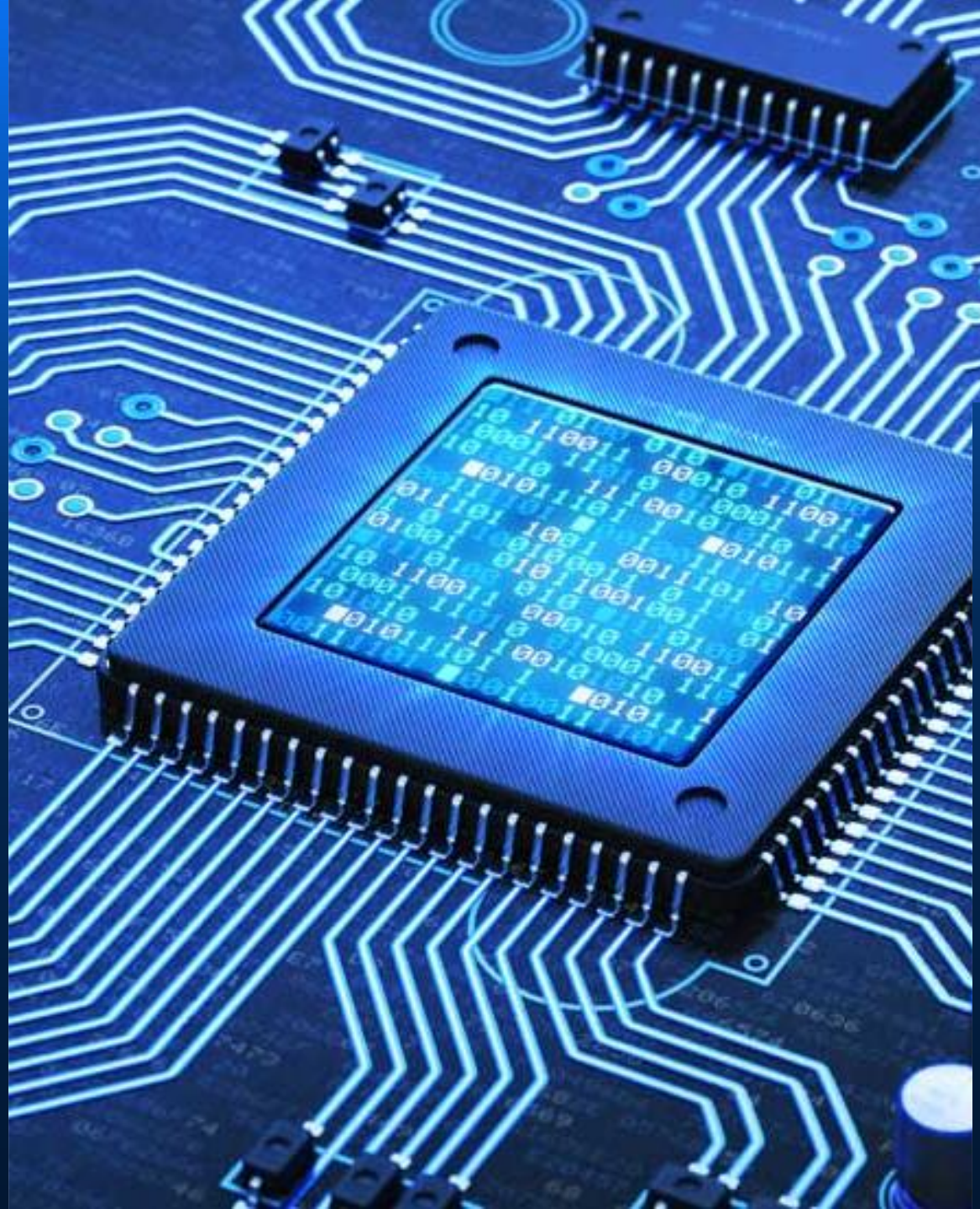


Computer Architecture

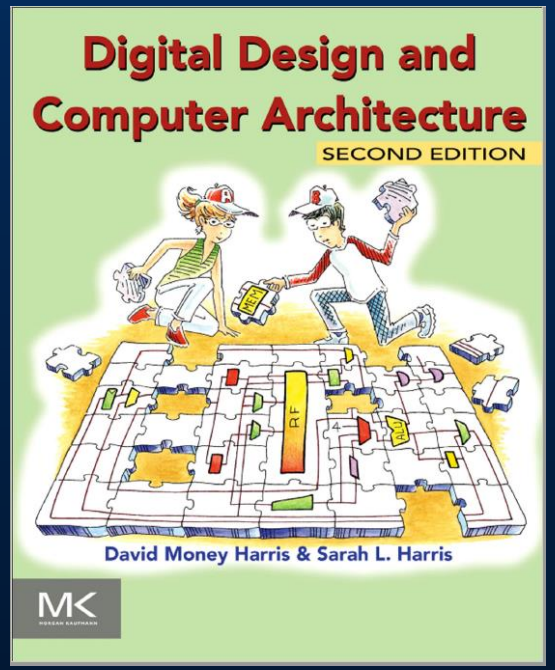
Session 6

By: Eng. Abdulkareem Mohammed

Senior HW Digital Design Engineer

This presentation is prepared from:

Digital Design and Computer Architecture, 2nd Edition.
David Harris and Sarah L. Harris.



Agenda

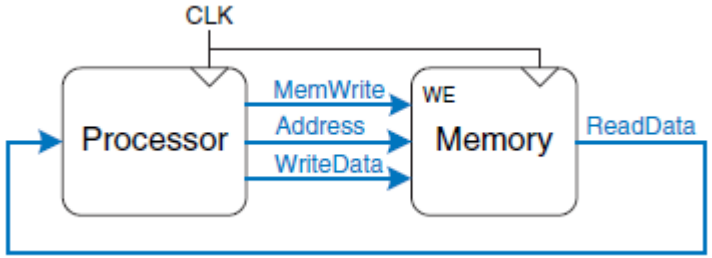
Chapter 8: Memory and I/O Systems

- # 1 Introduction
- # 2 Memory System Performance Analysis
- # 3 Caches
- # 4 Virtual Memory

1. Introduction

1. Introduction

- Computer performance depends on its memory system, the input/output (I/O) devices (monitors, keyboards, printers, …) and the processor microarchitecture.
- We assumed that the memory could be accessed in a single clock cycle, which is true only for a very small memory or a very slow processor!
- DRAM memories are currently 10 to 100 times slower than processors.



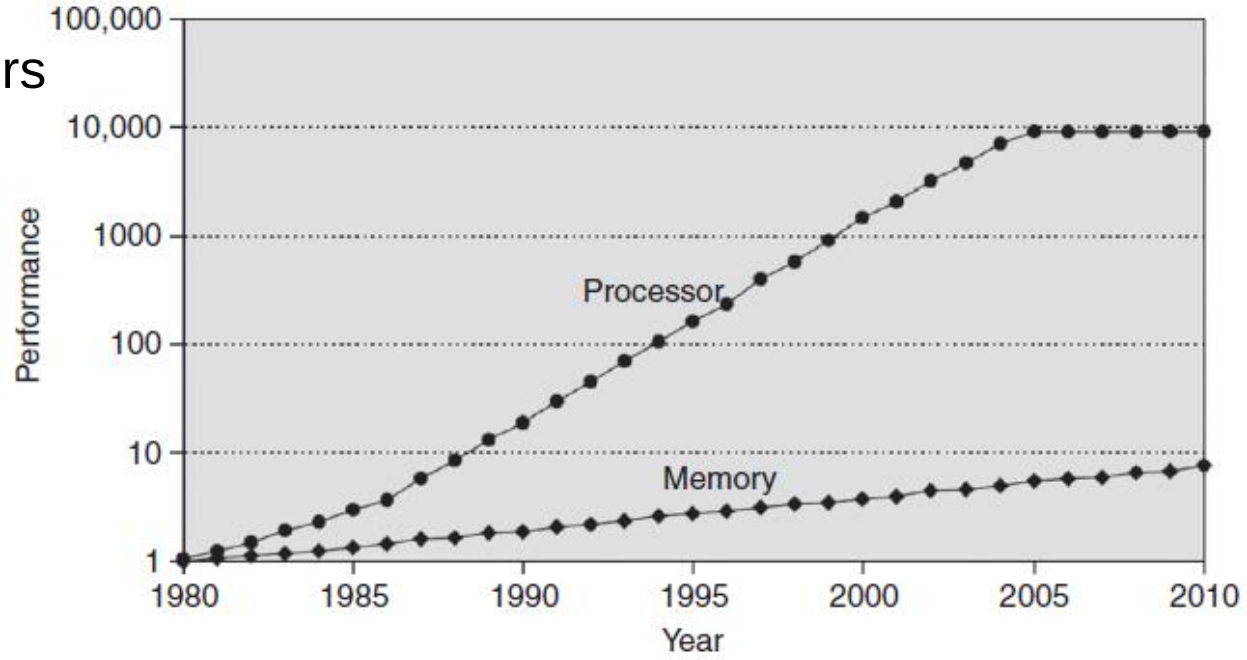
- The processor communicates with the memory system over a memory interface.
- About the simple memory interface, the processor sends an address over the Address bus.
  For a read, MemWrite is 0 and the memory returns the data on the ReadData bus.
  For a write, MemWrite is 1 and the processor sends data to memory on the WriteData bus.

1. Introduction

- Memory systems use a hierarchy of storage to quickly access the most commonly used data while still having the capacity to store large amounts of data.
- Computer memories are primarily built from dynamic RAM (DRAM) and static RAM (SRAM).
- Computer memory systems uses a fast small cheap memory and a slow large cheap memory.
- The fast small memory stores the most commonly used data and instructions, so on average the memory system appears **fast**.
- The slow large memory stores the remainder of data/instructions, so the overall capacity is **large**.
- The combination of two cheap memories is much less expensive than a single large fast memory.
- These principles extend to using an entire hierarchy of memories of increasing capacity and decreasing speed.

1. Introduction

- Computer memory is generally built from DRAM chips.
- In 2012, a typical PC had a main memory of 4 to 8 GB of **DRAM**, and costs about **$10/GB**.
- DRAM prices have declined at about 25% per year, and memory capacity has grown at the same rate, so the total cost of the memory has remained approximately constant.
- DRAM speed has improved by only about 7% per year, whereas processor performance has improved at a rate of 25 to 50% per year.
- Recently, DRAM access time is one to two orders of magnitude longer than the processor cycle time (tens of nanoseconds, compared to less than one nanosecond).

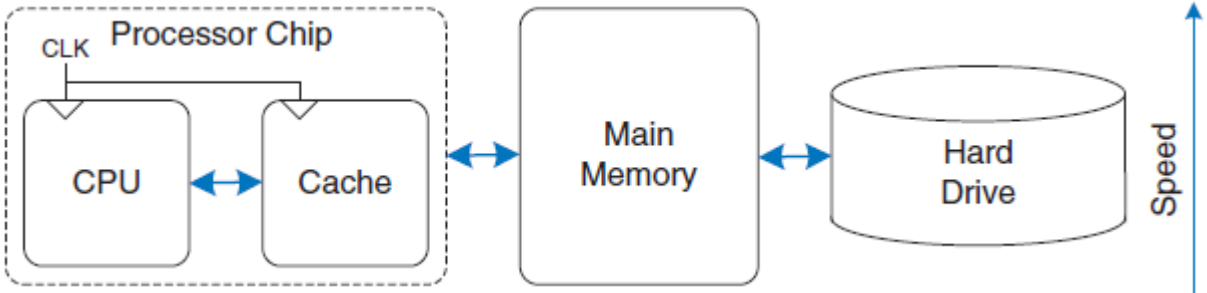


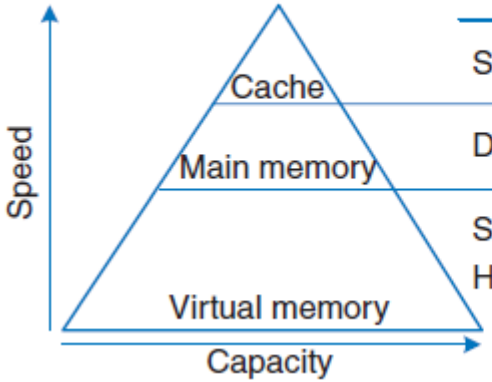
1. Introduction

- **Cache** is used to store the most commonly used instr and data in a faster smaller memory.
- The cache is usually built out of **SRAM** on the same chip as the processor.
- The cache speed is comparable to the processor speed, as the on-chip memory eliminates delays caused by traveling to and from a separate chip, and SRAM is faster than DRAM
- In 2012, on-chip SRAM costs were about **$10,000/GB**, but the cache is relatively small (kilobytes to several megabytes), so the overall cost is low.
- If the processor requests data that is available in the cache, it is returned quickly (cache hit). Otherwise, the processor retrieves the data from main memory (DRAM) (cache miss).
- **Hard drive** (third level in the hierarchy) is used to store data that does not fit in main memory.
- In 2012, a hard disk drive (**HDD**) built using magnetic storage, cost less than **$0.10/GB** and had an access time of about 10 ms.

1. Introduction

- **Solid state drives** (SSDs) built using flash memory technology, are a common alternate to HDDs.
- **SSDs** overcome some mechanical failures of HDDs, but they cost ten times as much at **$1/GB**.
- The hard drive provides more capacity than actually exists in the main memory (**virtual memory**).
- The processor first seeks data in a small fast **cache** that is usually located on the same chip. If the data doesn't exit, the processor then looks in **main memory**. If not there either, the processor fetches the data from **virtual memory** on the large slow hard disk.





Technology	Price / GB	Access Time (ns)	Bandwidth (GB/s)
SRAM	$10,000	1	25+
DRAM	$10	10–50	10
SSD	$1	100,000	0.5
HDD	$0.1	10,000,000	0.1

Agenda

Chapter 8: Memory and I/O Systems

- # 1 Introduction
- # 2 Memory System Performance Analysis
- # 3 Caches
- # 4 Virtual Memory

2 Memory System Performance Analysis

2 Memory System Performance Analysis

- Memory system performance metrics are miss rate or hit rate and average memory access time.
- Miss and hit rates are calculated as:

$$\text{Miss Rate} = \frac{\text{Number of misses}}{\text{Number of total memory accesses}} = 1 - \text{Hit Rate}$$

$$\text{Hit Rate} = \frac{\text{Number of hits}}{\text{Number of total memory accesses}} = 1 - \text{Miss Rate}$$

- Average memory access time (AMAT): the average time a processor must wait for memory per load or store instruction.

$$\text{AMAT} = t_{cache} + MR_{cache}\,(t_{MM} + MR_{MM} t_{VM})$$

where t_{cache}, t_{MM}, and t_{VM} are the access times of the cache, main memory, and virtual memory, and MR_{cache} and MR_{MM} are the cache and main memory miss rates, respectively.

Agenda

Chapter 8: Memory and I/O Systems

- # 1 Introduction
- # 2 Memory System Performance Analysis
- # 3 Caches
- # 4 Virtual Memory

3 Caches

3 Caches

- The computer system designer must choose which subset of the main memory is kept in the cache, as the capacity of the cache (C) is smaller than that of main memory.
- When the processor attempts to access data, it first checks the cache for the data. If the cache hits, the data is available immediately. If the cache misses, the processor fetches the data from main memory and places it in the cache for future use.
- To accommodate the new data, the cache must replace old data.
- Caches use spatial and temporal locality to predict what data will be needed next.
- Caches are specified by their capacity (C), number of sets (S), block size (b), number of blocks (B), and degree of associativity (N).
- Although we focus on data cache loads, the same is applied for fetches from an instruction cache.

Agenda

8.3: Caches

1. 1 What Data is Held in the Cache?
2. 2 How is Data Found?
3. 3 What Data is Replaced?
4. 4 Advanced Cache Design
5. 5 The Evolution of MIPS Caches

Agenda

8.3: Caches

1. 1 What Data is Held in the Cache?
2. 2 How is Data Found?
3. 3 What Data is Replaced?
4. 4 Advanced Cache Design
5. 5 The Evolution of MIPS Caches

3 Caches

What Data is Held in the Cache?

- The cache must guess what data will be needed based on the past pattern of memory accesses.
- In particular, the cache exploits temporal and spatial locality to achieve a low miss rate.
- **Temporal locality:** If a variable is used in a program, the same variable is likely to be used again .
- **Spatial locality:** If an element in an array is used, other elements in the same array are also likely to be used .
- Therefore, when the cache fetches one word from memory, it may also fetch several adjacent words (cache block/cache line).
- The number of words in the cache block, b, is called **the block size**.
- A cache of capacity C contains B = C/b blocks.

Agenda

8.3: Caches

- # 1 What Data is Held in the Cache?
- # 2 How is Data Found?
- # 3 What Data is Replaced?
- # 4 Advanced Cache Design
- # 5 The Evolution of MIPS Caches

3 Caches

How is Data Found?

- A cache is organized into **S** sets, each set holds one or more blocks of data.
- Each memory address in main memory **maps** to exactly one set in the cache.
- Some of the address bits are used to determine which cache set contains the data. If the set contains more than one block, the data may be kept in any of the blocks in the set.
- Caches are categorized based on the number of blocks in a set.
- In a direct mapped cache, each set contains exactly one block, so the cache has S = B sets. Thus, a particular main memory address maps to a unique block in the cache.
- In an N-way set associative cache, each set contains N blocks. The address still maps to a unique set, with S = B/N sets. But the data from that address can go in any of the N blocks in that set.

3 Caches

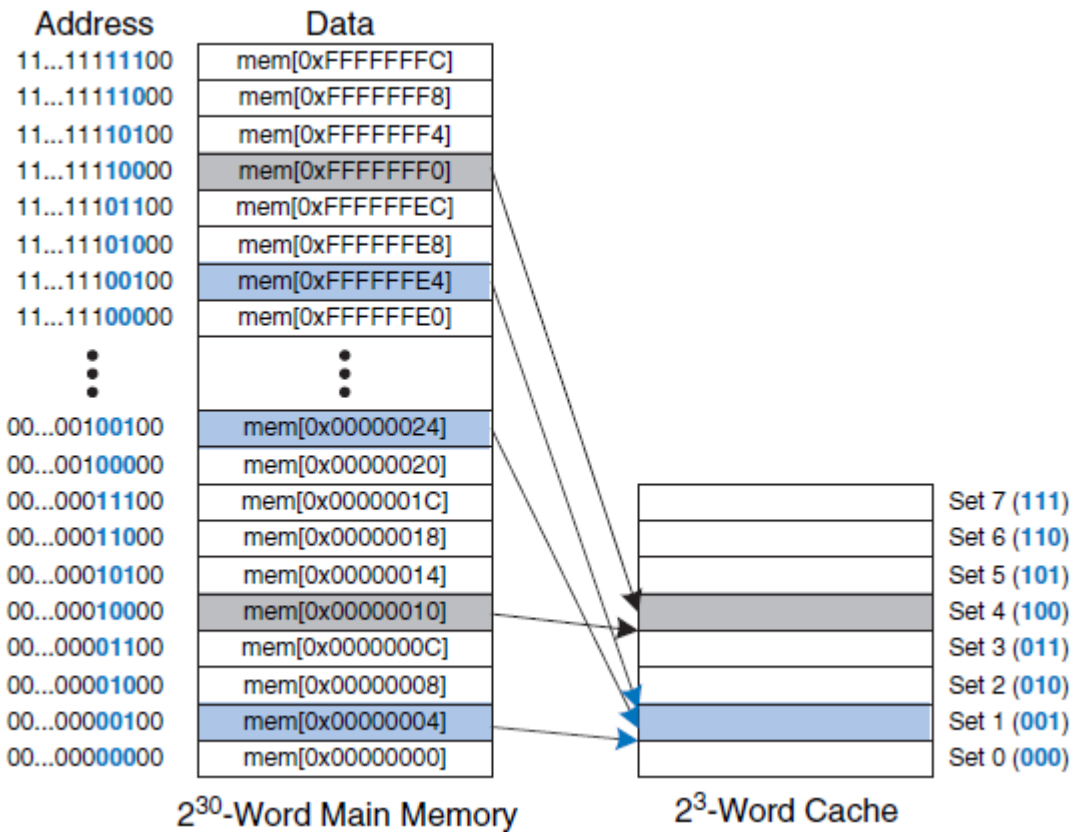
How is Data Found?

- A fully associative cache has only S = 1 set.
- Data can go in any of the B blocks in the set.
- A fully associative cache is considered as a B-way set associative cache.
- To illustrate these cache organizations, we will consider a MIPS memory system with 32-bit addresses and 32-bit words.
- The memory is byte addressable, and each word is four bytes, so the memory consists of 230 words aligned on word boundaries.
- We analyze caches with an eight-word capacity (C) for the sake of simplicity.

3 Caches

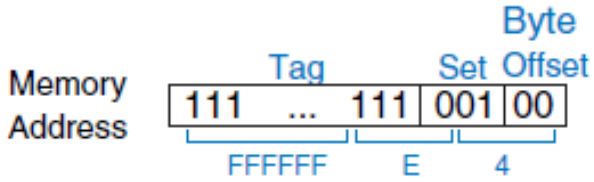
Direct Mapped Cache

- A direct mapped cache has one block in each set, so it is organized into S = B sets.
- An address in block 0 of main memory maps to set 0 of the cache, and so on until an address in block B−1 of main memory maps to block B−1 (7) of the cache.
- There are no more blocks of the cache, so the mapping wraps around, such that block B (8) of main memory maps to block 0 of the cache.
- The bottom two bits of the address are always 00, because they are word aligned.
- The data at addresses 0x00000004, 0x00000024, . . ., 0xFFFFFFE4 all map to set 1.



3 Caches

Direct Mapped Cache

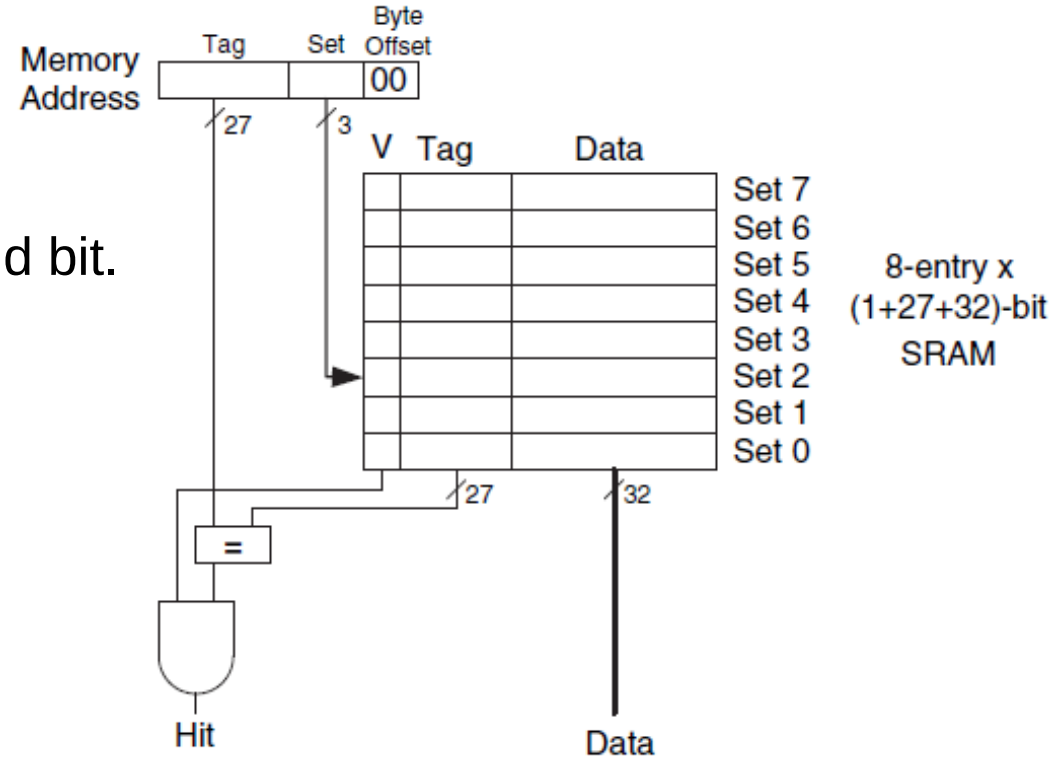


- The cache must also keep track of the address of the data actually contained in each set.
- The two least significant bits of the address (**byte offset**) indicate the byte within the word.
- The least significant bits of the address (**set**) specify which set holds the data.
- The remaining most significant bits (**tag**) and indicate which address is held in that set.
- The cache uses a **valid** bit for each set to indicate whether the set holds meaningful data.
  If the valid bit is 0, the contents are meaningless.

3 Caches

Direct Mapped Cache

- The cache is constructed as an 8-set SRAM.
- Each set contains 32 bits data, 27 bits of tag, and 1 valid bit.
- The cache is accessed using the 32-bit address.
- The two least significant bits (byte offset) are ignored.
- The next three bits (set bits) specify the cache set.
- A load instruction reads the specified entry from the cache and checks the tag and valid bits.

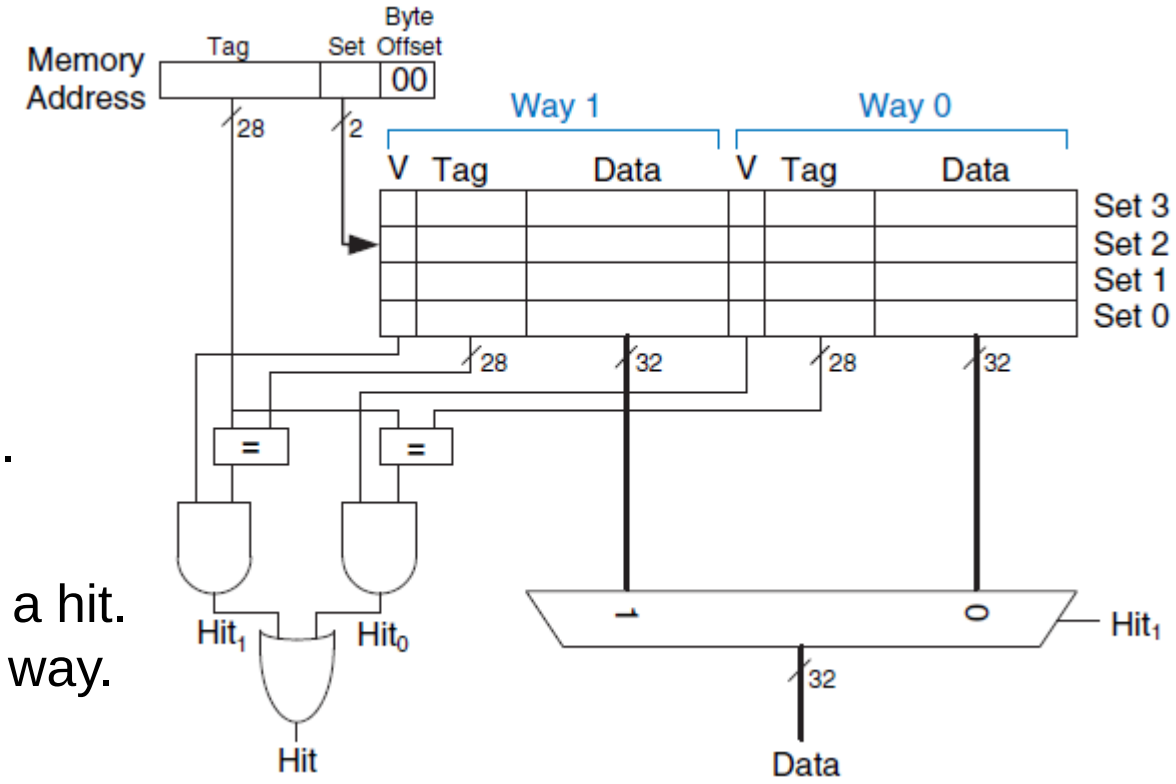


- If the tag matches the most significant 27 bits and the valid bit is 1, the cache hits and the data is returned to the processor. Otherwise, the cache misses and must fetch it from main memory.
- When two recently accessed addresses map to the same cache block, a conflict occurs, and the most recently accessed address evicts the previous one from the block.

3 Caches

Multi-way Set Associative Cache

- An N-way set associative cache reduces conflicts by providing N blocks in each set where data mapping to that set might be found.
- Each memory address maps to a specific set, but it can map to any one of the N blocks in the set.
- N is the degree of associativity of the cache.
- The C = 8-word, N= 2-way set associative cache now has only S = 4 sets rather than 8.
  Thus, only 2 set bits are used to select the set.
  The tag increases from 27 to 28 bits.
- Each way consists of a data and valid and tag bits.
- The cache reads blocks from both ways in the selected set and checks the tags and valid bits for a hit. If a hit occurs, a multiplexer selects data from that way.



3 Caches

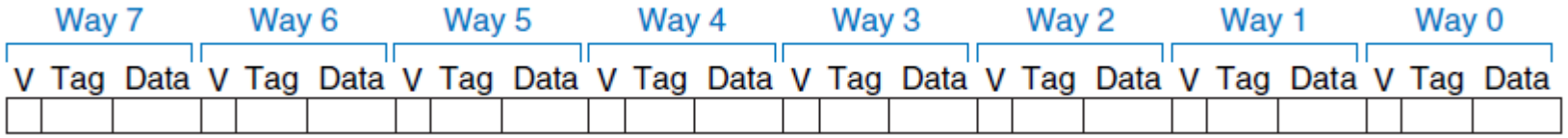
Multi-way Set Associative Cache

- Set associative caches generally have lower miss rates than direct mapped caches of the same capacity, as they have fewer conflicts.
- Set associative caches are usually slower and more expensive to build because of the output multiplexer and additional comparators.
- They also raise the question of which way to replace when both ways are full.
- Most commercial systems use set associative caches.

3 Caches

Fully Associative Cache

- A fully associative cache contains a single set with B ways, where B is the number of blocks.
- A memory address can map to a block in any of these ways.
- A fully associative cache is another name for a B-way set associative cache with one set.



- For a data request, eight tag comparisons must be made, because the data could be in any block.
- Similarly, an 8:1 multiplexer chooses the proper data if a hit occurs.
- Fully associative caches have **the fewest conflict misses** for a given cache capacity, but they require **more hardware** for additional tag comparisons.
- They are used with the small caches because of the large number of comparators.

3 Caches

Block Size

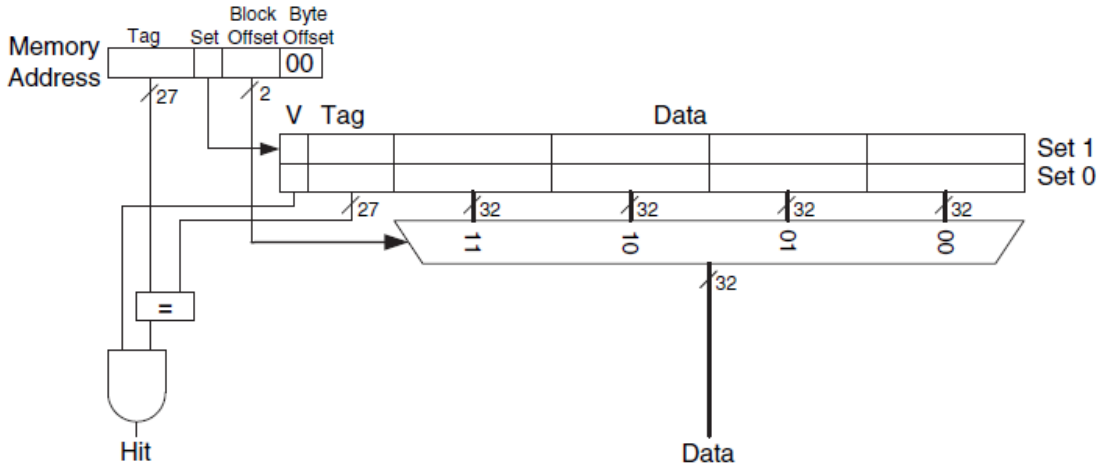
- The previous examples focused on the **temporal locality**, because the block size was one word.
- For **spatial locality**, a cache uses larger blocks to hold several consecutive words.
- When a miss occurs and the word is fetched into the cache, the adjacent words in the block are also fetched, so the subsequent accesses are more likely to **hit** because of spatial locality.

- However, a large block size means that the cache will have fewer blocks, increasing the miss rate.
- Moreover, it takes more time to fetch the missing cache block (**the miss penalty**) after a miss.
- If the adjacent words in the block are not accessed later, the effort of fetching them is wasted.
- Nevertheless, most real programs benefit from larger block sizes.

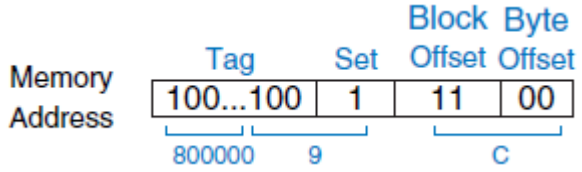
3 Caches

Block Size

- The C = 8-word direct mapped cache with a b = 4-word block size has only B = C/b = 2 blocks.
- A direct mapped cache has one block in each set, so this cache is organized as two sets. Thus, only 1-bit is used to select the set.
- A multiplexer is now needed to select the word within the block. It's controlled by 2 block offset bits.
- The tag is the most significant 27 address bits.
- Only one tag is needed for the entire block, because the words in the block are at consecutive addresses.



- Example: The cache fields for address 0x8000009C.
  The byte offset bits are always 0 for word accesses.
  The next 2-bits (block offset) indicate the word within the block.
  And the next bit indicates the set. The remaining 27 bits are the tag.



3 Caches

Putting it All Together

- Caches are organized as two-dimensional arrays. The rows are called sets, and the columns are called ways. Each entry in the array consists of a data block and its associated valid and tag bits.
- Caches are characterized by
  - capacity (**C**).
  - block size **b** (and number of blocks, **B** = C/b)
  - number of blocks in a set (**N**)
- Each address in memory maps to only one set but can be stored in any of the ways.

Organization	Number of Ways (N)	Number of Sets (S)
Direct Mapped	1	B
Set Associative	1 <N< B	B/N
Fully Associative	B	1

- Increasing the associativity **N** usually reduces the miss rate, but requires more tag comparators.
- Increasing the block size **b** reduces the miss rate (spatial locality), but decreases the number of sets in a fixed sized cache (more conflicts), and increases the miss penalty.

Agenda

8.3: Caches

- # 1 What Data is Held in the Cache?
- # 2 How is Data Found?
- # 3 Which Data is Replaced?
- # 4 Advanced Cache Design
- # 5 The Evolution of MIPS Caches

3 Caches

Which Data is Replaced?

- In a direct mapped cache, if a set is full, the block in that set is replaced with the new data.
- In set associative and fully associative caches, the cache must choose which block to replace. The temporal locality principle suggests to replace the least recently used block.
- Most associative caches have a least recently used (**LRU**) replacement policy.
- In a two-way set associative cache, a use bit (**U**) indicates which way within a set was least recently used. Each time one way is used, **U** will indicate the other way.
- For set associative caches with N ways, tracking the least recently used way is complicated.
- So, the ways are divided into two groups and **U** indicates which group was least recently used.
- The new block replaces a random block within the least recently used group (**pseudo-LRU**).

Agenda

8.3: Caches

- # 1 What Data is Held in the Cache?
- # 2 How is Data Found?
- # 3 What Data is Replaced?
- # 4 Advanced Cache Design
- # 5 The Evolution of MIPS Caches

3 Caches

Multiple-Level Caches

- Modern systems use multiple levels of caches to decrease memory access time.
- Large caches are beneficial because they are more likely to hold data with lower miss rates.
- However, large caches is slower than small ones.
- Modern systems often use at least two levels of caches.
- The first-level (L1) cache is small enough to provide a one- or two-cycle access time.
- The second-level (L2) cache is also SRAM but is larger and slower than the L1 cache.
- The processor first looks for the data in the L1 cache. If misses, the processor looks in L2 cache.
- If the L2 cache misses, the processor fetches the data from main memory.
- Many modern systems add even more levels of cache to the memory hierarchy, because accessing main memory is so slow.

3 Caches

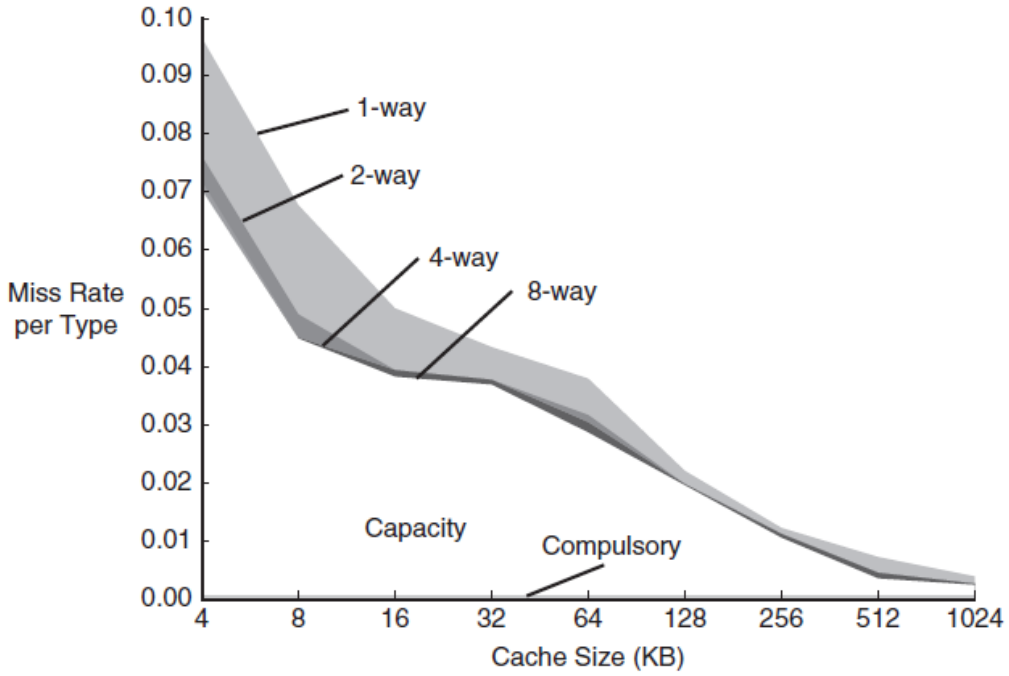
Reducing Miss Rate

- Cache misses can be reduced by changing capacity, block size, and/or associativity.
- The misses can be classified as **compulsory**, **capacity**, and **conflict**.
- Compulsory misses occur with the first request to a cache block, because the block must be read from memory regardless of the cache design.
- Capacity misses occur when the cache is too small to hold all concurrently used data.
- Conflict misses occur when several addresses map to the same set and evict needed blocks.
- Increasing cache capacity can reduce conflict and capacity misses, but it does not affect compulsory misses.
- Increasing block size could reduce compulsory misses (due to spatial locality) but might actually increase conflict misses (because more addresses would map to the same set and could conflict).
- Running benchmarks while varying cache parameters to evaluate the cache performance .

3 Caches

Reducing Miss Rate

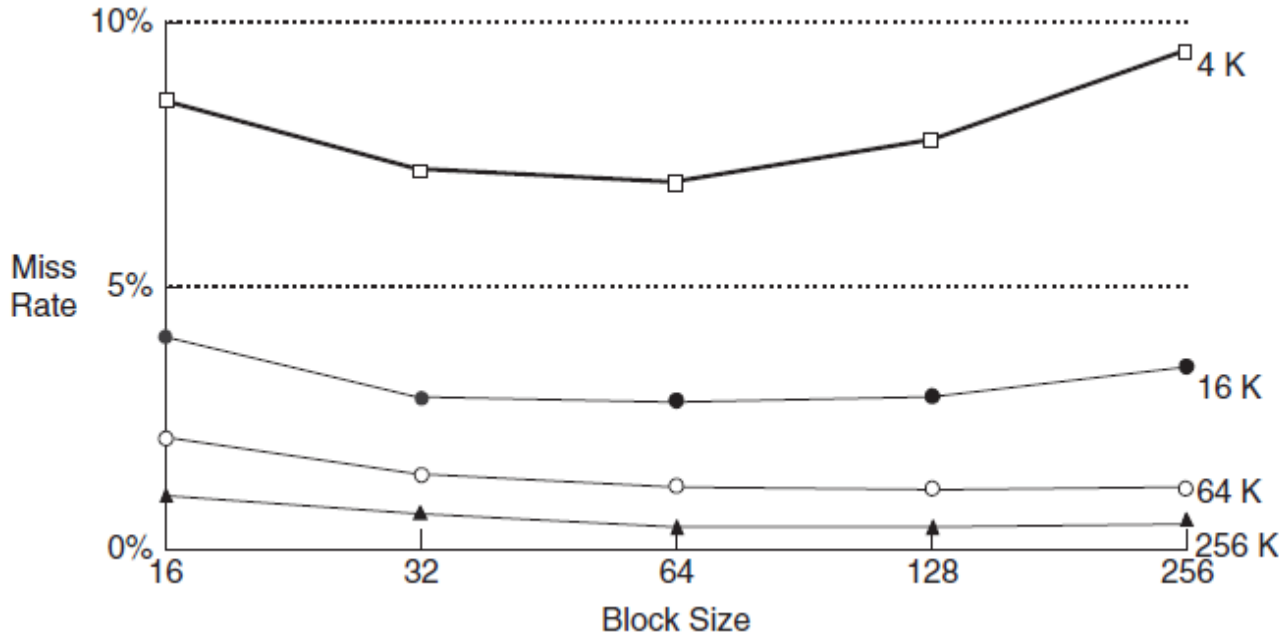
- Comparing the miss rate vs cache size and associativity degree using the SPEC2000 benchmark.
- This benchmark has a small number of compulsory misses, shown by the dark region near x-axis.
- As expected, when cache size increases, capacity misses decrease.
- Increasing associativity, especially for small caches, decreases the number of conflict misses shown along the top of the curve.
- Increasing associativity beyond four or eight ways provides only small decreases in miss rate.



3 Caches

Reducing Miss Rate

- As mentioned, miss rate can also be decreased by using larger block sizes (spatial locality).
- But as block size increases, the number of sets decreases, increasing the probability of conflicts.
- For small caches, such as the 4-KB cache, increasing the block size beyond 64 bytes increases the miss rate because of conflicts.
- For larger caches, increasing the block size after 64 bytes doesn't change the miss rate.
- However, large block sizes might still increase execution time because of the larger miss penalty.



3 Caches

Write Policy

- Memory stores, or writes, follow a similar procedure as loads.
- Upon a memory store, the processor checks the cache. If the cache misses, the cache block is fetched from main memory into the cache, and then the appropriate word in the cache block is written. If the cache hits, the word is simply written to the cache block.
- Caches are classified as either **write-through** or **write-back**.
- In a write-through cache, the data written to cache block is simultaneously written to main memory.
- In a write-back cache, a dirty bit (**D**) is associated with each cache block. D is 1 when the cache block has been written and 0 otherwise. Dirty cache blocks are written back to main memory only when they are evicted from the cache.
- A write-through cache requires no dirty bit but usually requires more main memory writes than a write-back cache. Modern caches are usually write-back, to reduce the main memory access time.

Agenda

8.3: Caches

1 What Data is Held in the Cache?

2 How is Data Found?

3 What Data is Replaced?

4 Advanced Cache Design

5 The Evolution of MIPS Caches

3 Caches

The Evolution of MIPS Caches

- The major trends are using multiple levels of cache, larger capacity, and increased associativity.
- These trends are driven by the growing disparity between CPU frequency and main memory speed and the decreasing cost of transistors.
- The growing difference between CPU and memory speeds requires a lower miss rate to avoid the main memory bottleneck, and the decreasing cost of transistors allows larger cache sizes.

Year	CPU	MHz	L1 Cache	L2 Cache
1985	R2000	16.7	none	none
1990	R3000	33	32 KB direct mapped	none
1991	R4000	100	8 KB direct mapped	1 MB direct mapped
1995	R10000	250	32 KB two-way	4 MB two-way
2001	R14000	600	32 KB two-way	16 MB two-way
2004	R16000A	800	64 KB two-way	16 MB two-way
2010	MIPS32 1074K	1500	32 KB	variable size

Agenda

Chapter 8: Memory and I/O Systems

- # 1 Introduction
- # 2 Memory System Performance Analysis
- # 3 Caches
- # 4 Virtual Memory

4 Virtual Memory

4 Virtual Memory

- Most modern computer systems use a hard drive made of magnetic or solid-state storage as the lowest level in the memory hierarchy.
- The hard drive provides a much larger capacity than main memory (DRAM) but slower.
- If many memory accesses involve the hard drive, performance will be bad.
- The hard disk contains one or more rigid **disks** or platters, each of which has a read/write **head** on the end of a long triangular **arm**.
- The head moves to the correct location on the disk and reads or writes data magnetically as the disk rotates below it.
- The head takes several milliseconds to seek the correct location on the disk, which is much slower than the processor.

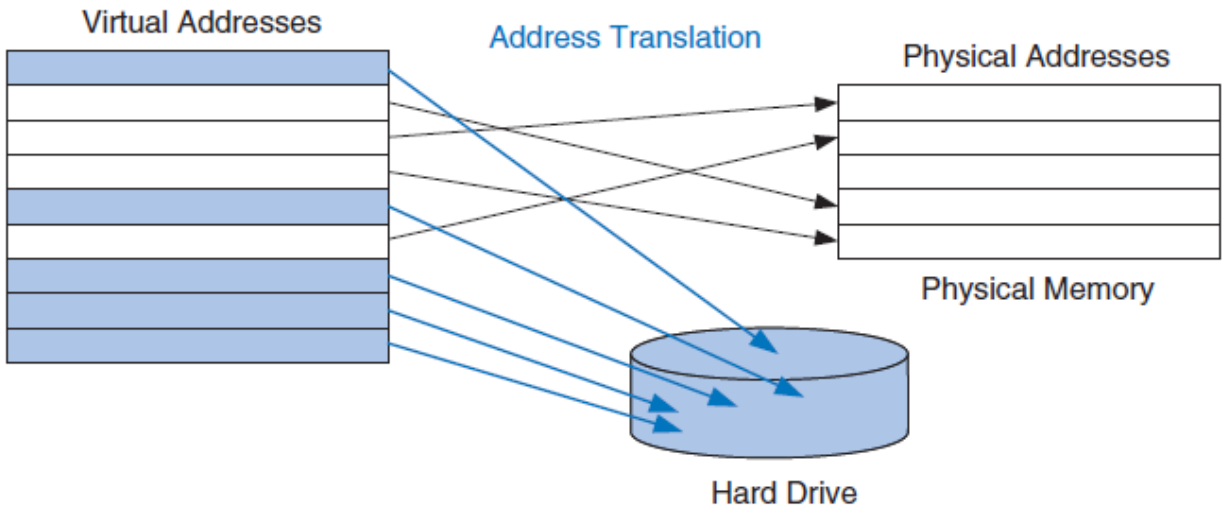


4 Virtual Memory

- Adding a hard drive to the memory hierarchy is to give the illusion of a very large cheaper memory while still providing the speed of faster memory for most accesses.
- Example: A computer with only 128 MB of DRAM **(physical memory)** could effectively provide 2GB hard drive memory **(virtual memory)**.
- Programs can access data anywhere in virtual memory, so they must use virtual addresses that specify the location in virtual memory.
- The physical memory holds a subset of most recently accessed virtual memory.
- In this way, physical memory acts as a cache for virtual memory.
- Thus, most accesses hit in physical memory at the speed of DRAM, yet the program enjoys the capacity of the larger virtual memory.

4 Virtual Memory

- Virtual memory is divided into **virtual pages**, typically 4 KB in size.
- Physical memory is divided into **physical pages** of the same size.
- A virtual page may be located in physical memory (DRAM) or on the hard drive.
- Some virtual pages are present in physical memory, and some are located on the hard drive.
- The **address translation** process is to determine the physical address from the virtual address.
- If the processor attempts to access a virtual address that is not in physical memory, a page fault occurs, and the operating system loads the page from the hard drive into physical memory.



4 Virtual Memory

- To avoid page faults caused by conflicts, any virtual page can map to any physical page. The physical memory behaves as a fully associative cache for virtual memory.
- Like the fully associative cache, each physical page in a virtual memory will need a comparator to check the most significant virtual address bits against a tag to determine whether the virtual page maps to that physical page.
- Instead, the virtual memory system uses **a page table** to perform address translation.
- A page table contains an entry for each virtual page, indicating its location in physical memory or that it is on the hard drive.

4 Virtual Memory

- Each load or store instruction requires a page table access followed by a physical memory access.
- The page table access translates the **virtual address** used by the program to **a physical address**.
- The physical address is then used to actually read or write the data.
- The page table is usually so large that it is located in physical memory.
- Hence, each load or store involves two physical memory accesses: a page table access, and a data access.
- To speed up address translation, **a translation lookaside buffer** (TLB) caches the most commonly used page table entries.

Agenda

8.4: Virtual Memory

- # 1 Address Translation
- # 2 The Page Table
- # 3 The Trans Lookaside Buf
- # 4 Memory Protection
- # 5 Replacement Policies
- # 6 Multilevel Page Tables

Agenda

8.4: Virtual Memory

- # 1 Address Translation
- # 2 The Page Table
- # 3 The Trans Lookaside Buf
- # 4 Memory Protection
- # 5 Replacement Policies
- # 6 Multilevel Page Tables

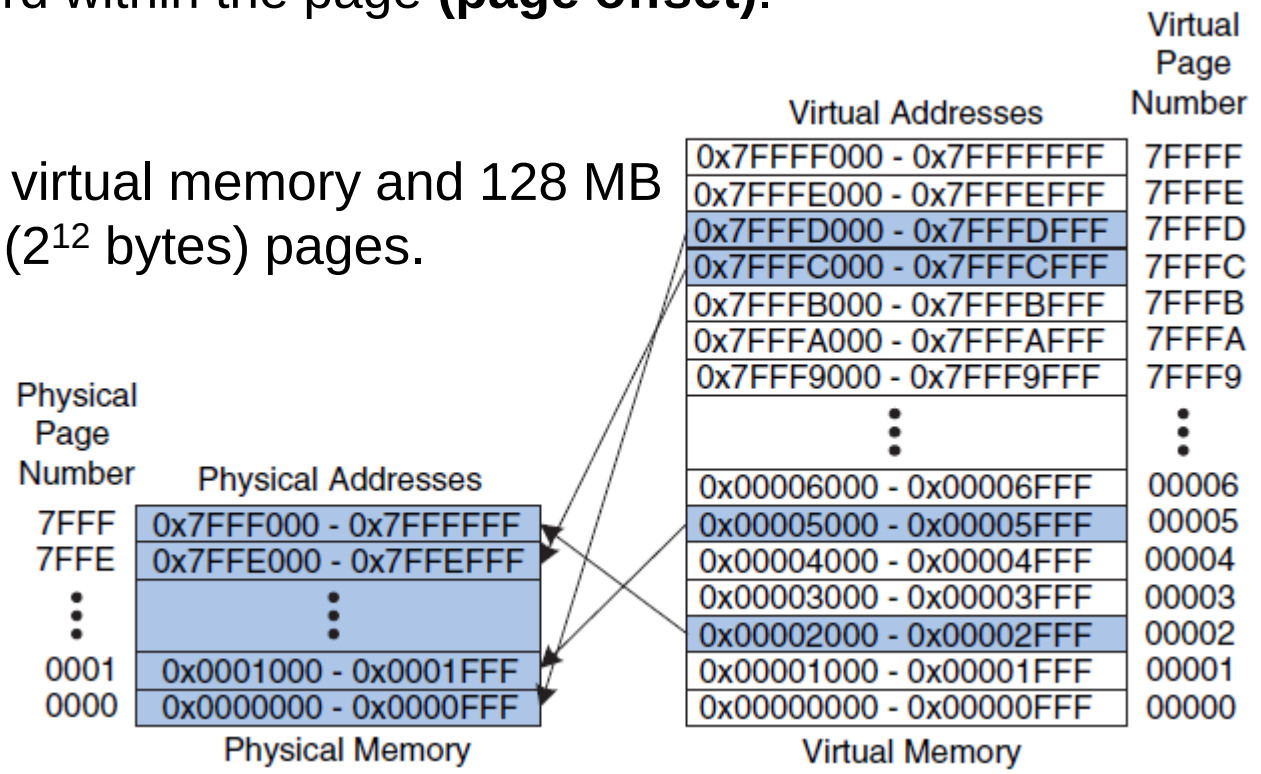
4 Virtual Memory

1. Address Translation

- The virtual memory and physical memory are divided into pages.
- The most significant bits of the virtual or physical address specify the virtual or physical page number. The least significant bits specify the word within the page **(page offset)**.

Example:

- A virtual memory system with 2 GB (2^{31}-byte) of virtual memory and 128 MB (2^{27}-byte) of physical memory divided into 4-KB (2^{12} bytes) pages.
- There are $2^{31}/2^{12} = 2^{19}$ virtual pages and $2^{27}/2^{12} = 2^{15}$ physical pages.
- Thus, the virtual and physical page numbers are 19 and 15 bits, respectively.
- Physical memory can only hold up to 1/16th of the virtual pages.

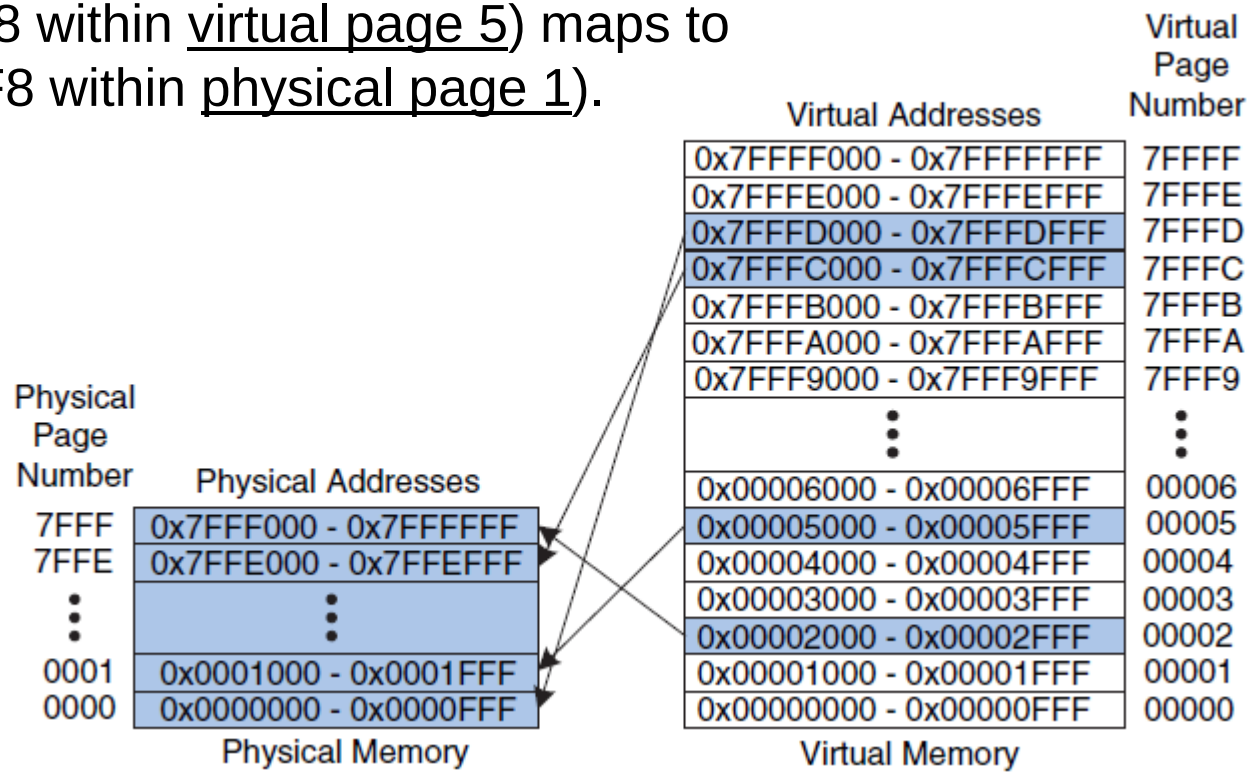


4 Virtual Memory

1. Address Translation

Example:

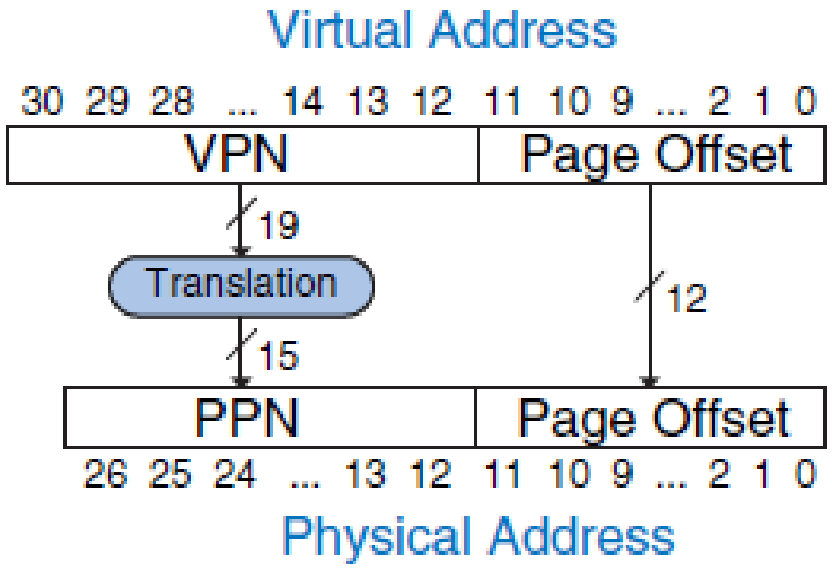
- Virtual page 0x7FFFC mapping to physical page 0x7FFE, and so on.
- Virtual address 0x000053F8 (an offset of 0x3F8 within virtual page 5) maps to physical address 0x00013F8 (an offset of 0x3F8 within physical page 1).
- The least significant 12 bits of the virtual and physical addresses are the same (0x3F8) and specify the **page offset**.



4 Virtual Memory

1. Address Translation

- Only the page number should be translated to obtain the physical address from the virtual address.
- The least significant 12 bits indicate the page offset and require no translation.
- The upper 19 bits of the virtual address specify the virtual page number (VPN) and are translated to a 15-bit physical page number (PPN).



Agenda

8.4: Virtual Memory

1 Address Translation

2 The Page Table

3 The Trans Lookaside Buf

4 Memory Protection

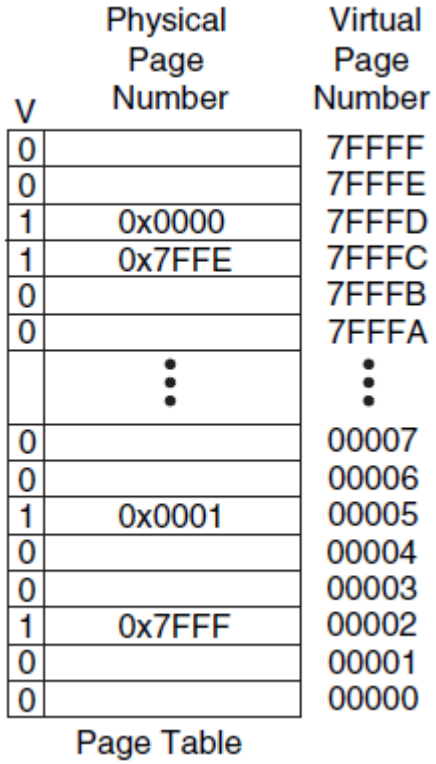
5 Replacement Policies

6 Multilevel Page Tables

4 Virtual Memory

2. The Page Table

- The page table is used to translate virtual addresses to physical addresses.
- The page table contains an entry (a physical page number and a valid bit) for each virtual page.
- If the valid bit is 1, the virtual page maps to the physical page specified in the entry. Otherwise, the virtual page is found on the hard drive.
- Because the page table is so large, it is stored in **physical memory**.
- The page table is indexed with the virtual page number (VPN).
- The page table can be stored anywhere in physical memory by the OS.
- The processor typically uses a dedicated register (**the page table register**), to store the base address of the page table in physical memory.



Page Table

4 Virtual Memory

2. The Page Table

To perform a load or store:

- The processor must first translate the virtual address to a physical address and then access the data at that physical address.
- The processor extracts the **virtual page number** from the virtual address and adds it to the page table register to find the physical address of the page table entry.
- The processor then reads this entry from physical memory to obtain the **physical page number**.
- If valid, it merges this physical page number with the page offset to get the **physical address**.
- Finally, it reads or writes data at this physical address.
- Because the page table is stored in physical memory, each load or store involves two physical memory accesses.

Agenda

8.4: Virtual Memory

1 Address Translation

2 The Page Table

3 The Trans Lookaside Buf

4 Memory Protection

5 Replacement Policies

6 Multilevel Page Tables

4 Virtual Memory

3. The Translation Lookaside Buffer

- Virtual memory would have a bad performance impact if it required a page table read on every load or store, doubling the delay of loads and stores.
- The temporal and spatial locality of data accesses and the large page size mean that many consecutive loads or stores are likely to use the same page.
- In general, the processor can keep the last several page table entries **in a small cache** called a **translation lookaside buffer** (TLB). The processor “looks aside” to find the translation in the TLB before having to access the page table in physical memory.
- In real programs, most accesses hit in the TLB, avoiding the time-consuming page table reads from physical memory.

4 Virtual Memory

3. The Translation Lookaside Buffer

- A TLB is organized as **a fully associative cache** and typically holds 16 to 512 entries.
- Each TLB entry holds a virtual page number and its corresponding physical page number.
- The TLB is accessed using the virtual page number. If the TLB hits, it returns the corresponding physical page number. Otherwise, the processor must read the page table in physical memory.
- The TLB is designed to be small enough that it can be accessed in less than one cycle. Even so, TLBs typically have a hit rate of greater than 99%.
- The TLB decreases the number of memory accesses required for most load or store instructions from two to one.

Agenda

8.4: Virtual Memory

1 Address Translation

2 The Page Table

3 The Trans Lookaside Buf

4 Memory Protection

5 Replacement Policies

6 Multilevel Page Tables

4 Virtual Memory

4 Memory Protection

- An important reason to use virtual memory is to protect the concurrently running programs, so that no program can crash or hijack another program.

- Virtual memory systems provide protection by giving each program its own virtual address space.
- Each program can use as much memory as it wants in that virtual address space, but only a portion of the virtual address space is in physical memory at any given time without knowing where other programs are physically located.

- In some cases, multiple programs access common instructions or data.
- The operating system adds control bits to each page table entry to determine which programs can write to the shared physical pages.

Agenda

8.4: Virtual Memory

- # 1 Address Translation
- # 2 The Page Table
- # 3 The Trans Lookaside Buf
- # 4 Memory Protection
- # 5 Replacement Policies
- # 6 Multilevel Page Tables

4 Virtual Memory

5 Replacement Policies

- Virtual memory systems use **write-back** and **an approximate least recently used** (LRU) replacement policy.
- The physical page is written back to the hard drive only when it is evicted from physical memory.
- Writing the physical page back to the hard drive and reloading it with a different virtual page is called **paging**, and the hard drive in a virtual memory system is sometimes called **swap space**.
- The processor pages out one of the least recently used physical pages when a page fault occurs, then replaces that page with the missing virtual page.
- For the replacement policies, each page table entry contains: a dirty bit **D** and a use bit **U**.
- The dirty bit is 1 if the physical page was changed since it was read from the hard drive.
- When a physical page is paged out, it needs to be written back to the hard drive only if its dirty bit is 1; otherwise, the hard drive already holds an exact copy of the page.

4 Virtual Memory

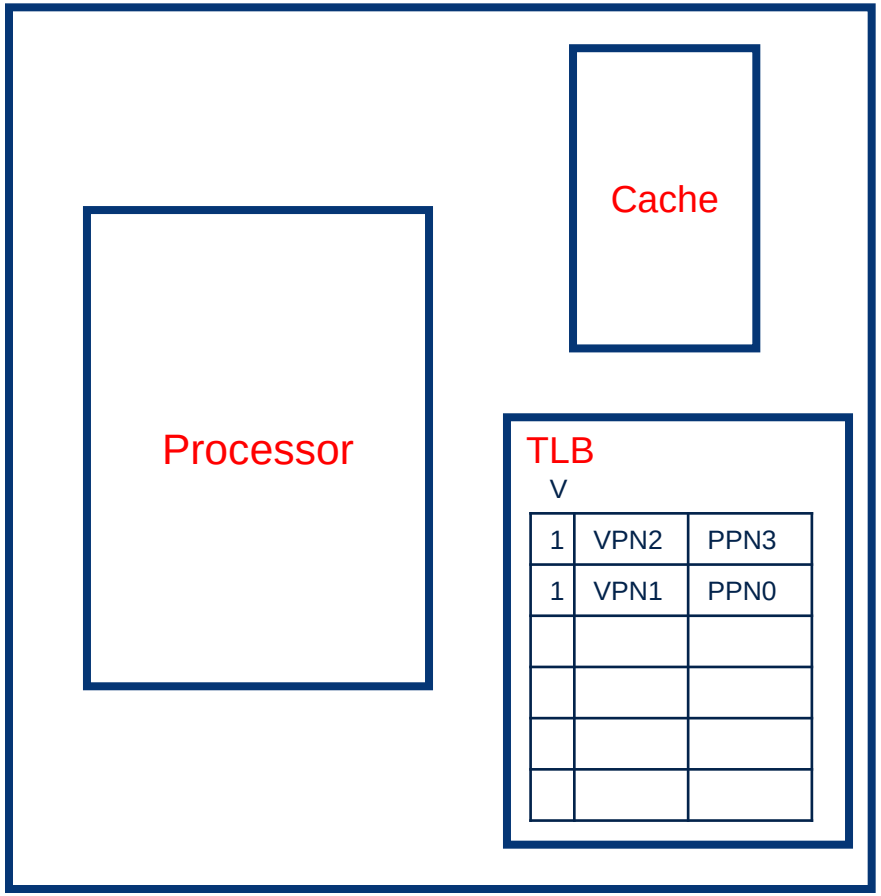
5 Replacement Policies

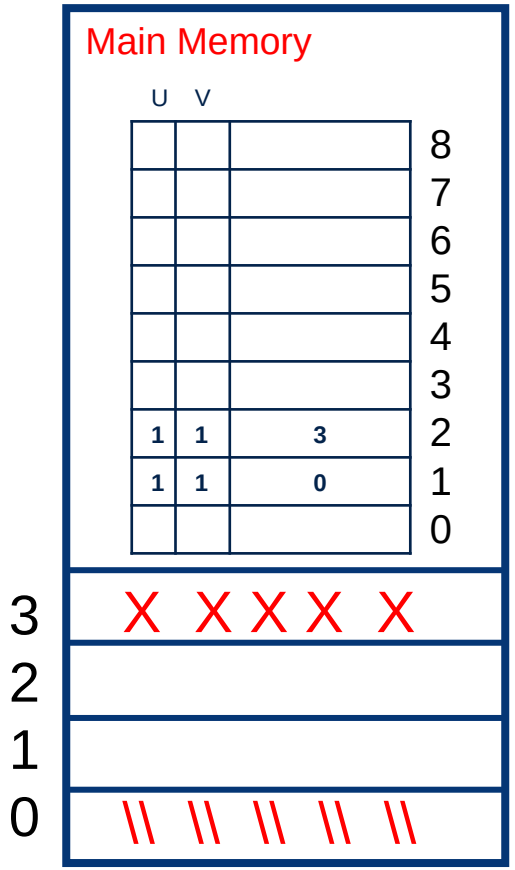
- The use bit is 1 if the physical page has been accessed recently.
- As in a cache system, exact LRU replacement would be impractically complicated.
- Instead, the OS approximates LRU replacement by periodically resetting all the use bits in the page table. When a page is accessed, its use bit is set to 1.
- Upon a page fault, the OS finds a page with U = 0 to page out of physical memory. Thus, it does not necessarily replace the least recently used page, just one of the least recently used pages.

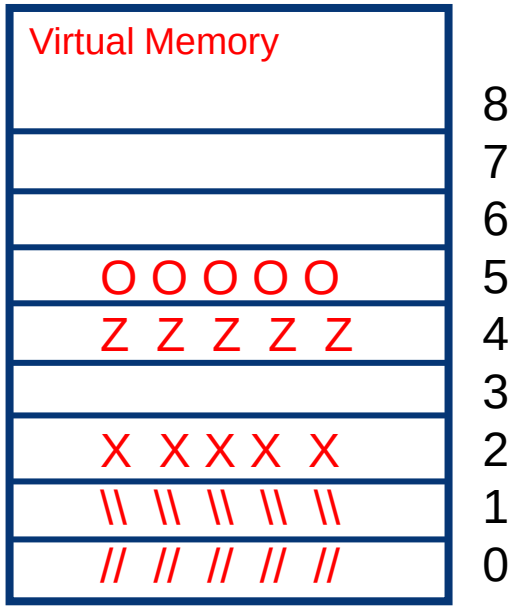
4 Virtual Memory

Example:

- Assume the following contents of page table and TLB.



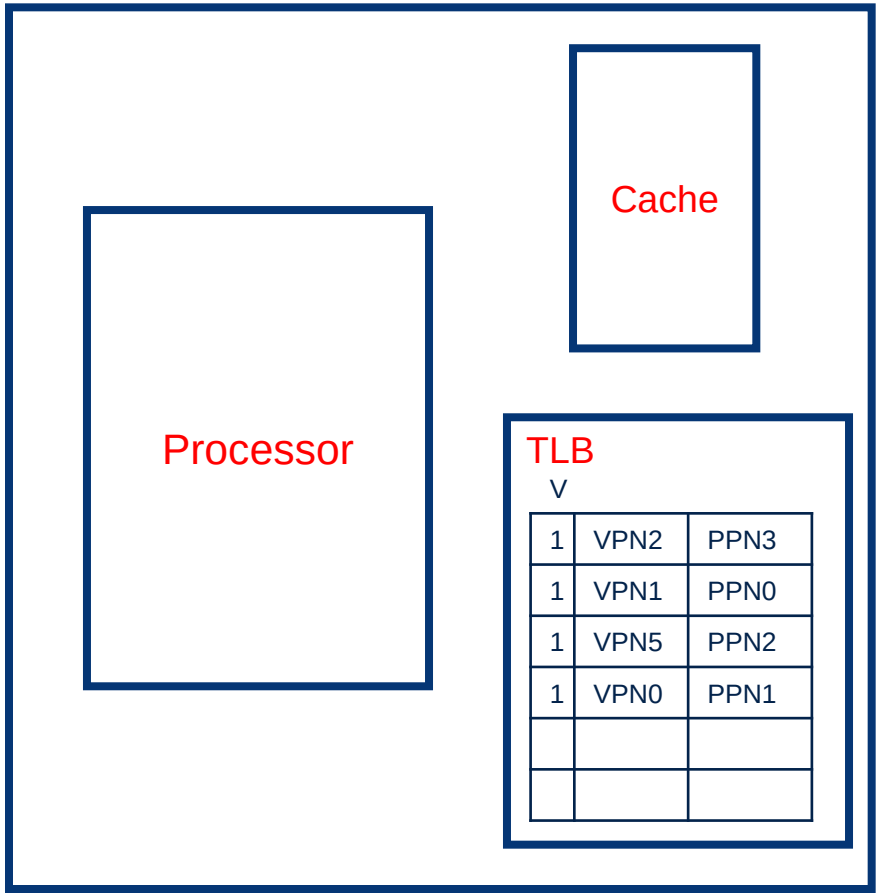


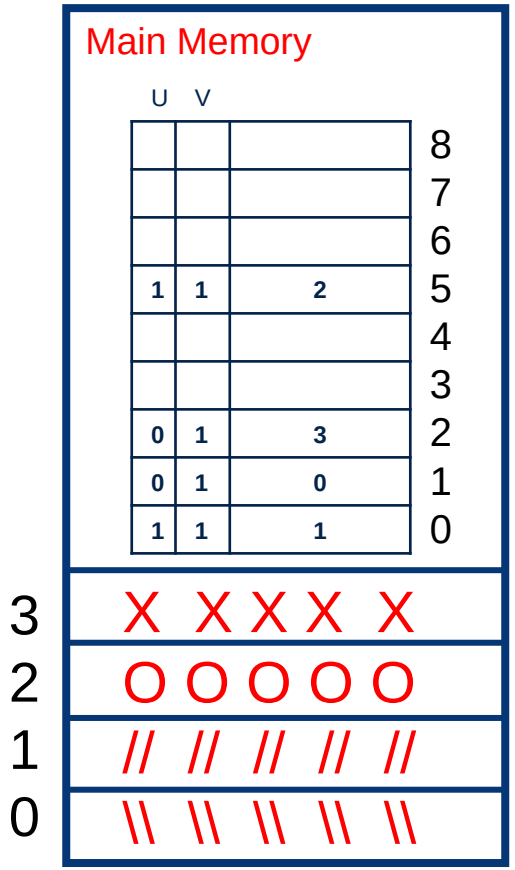


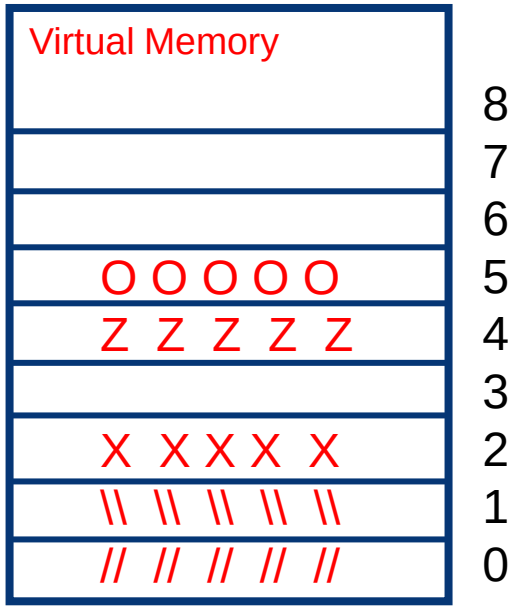
4 Virtual Memory

Example:

- Assume the update for the contents is reset at the current moment.



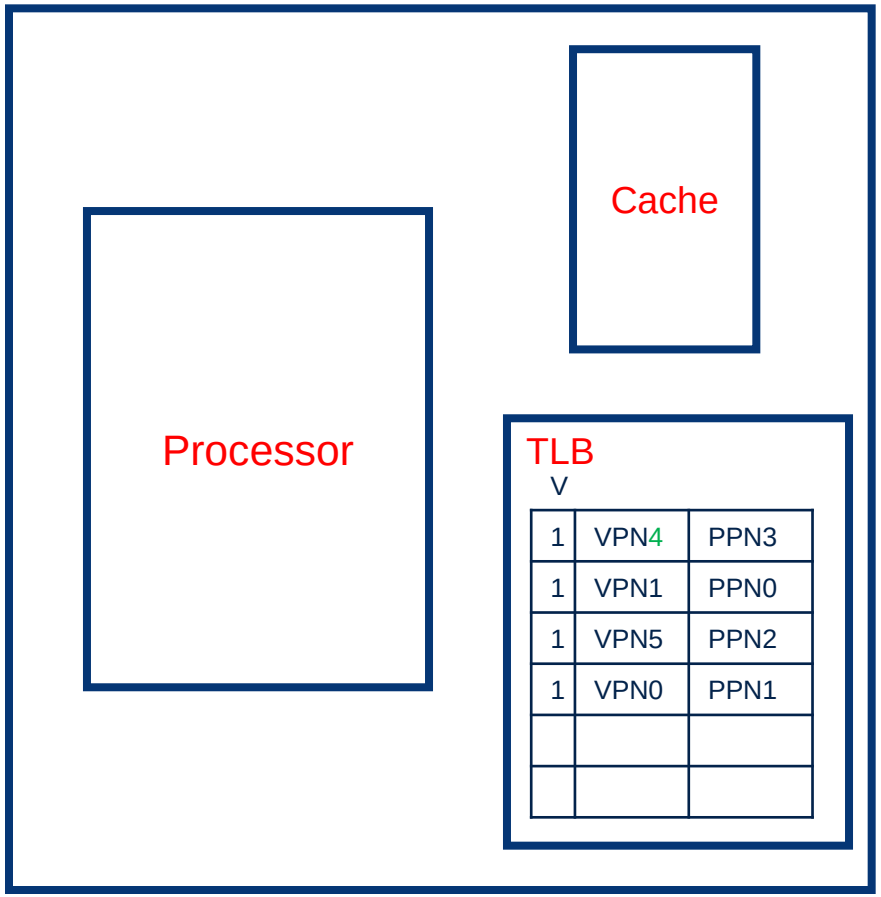


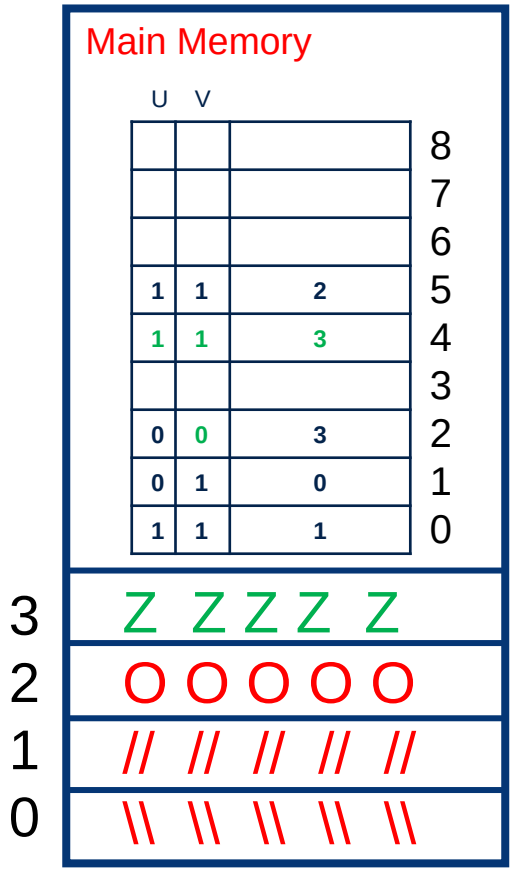


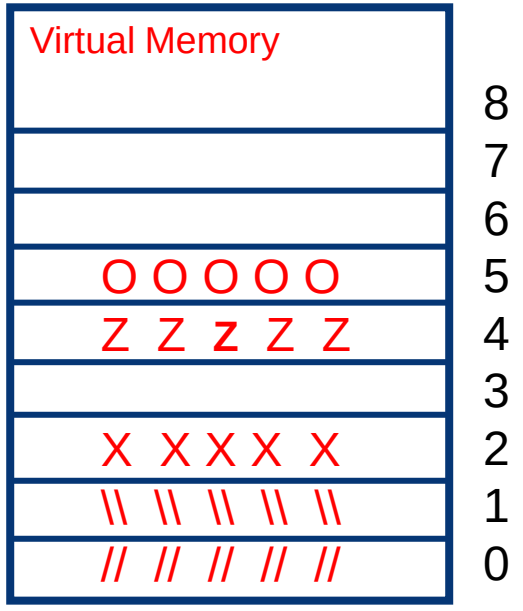
4 Virtual Memory

Example:

- Assume the processor wants to access now page #4 (VA), then page #1 or page #2 will be removed from the main memory.







Agenda

8.4: Virtual Memory

1 Address Translation

2 The Page Table

3 The Trans Lookaside Buf

4 Memory Protection

5 Replacement Policies

6 Multilevel Page Tables

4 Virtual Memory

6 Multilevel Page Tables

- Page tables can occupy a large amount of physical memory.

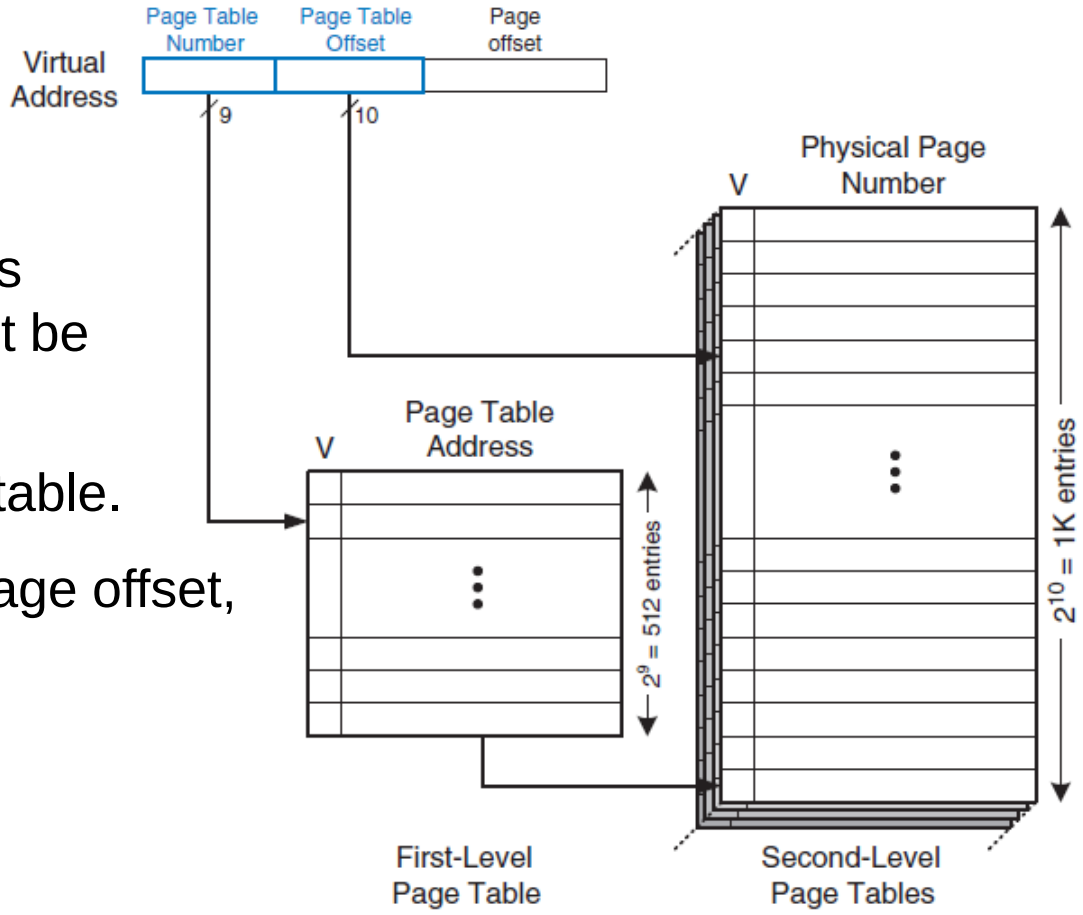
Example:

- A page table for a 2 GB virtual memory with 4 KB pages would need 2^{19} entries. If each entry is 4 bytes, the page table is $2^{19} \times 2^{2}$ bytes = 2^{21} bytes = 2 MB.
- To conserve physical memory, page tables can be broken up into multiple (usually two) levels.
- The first-level page table is always kept in physical memory. It indicates where small second-level page tables are stored in virtual memory.
- The second-level page tables each contain the actual translations for a range of virtual pages. If a particular range of translations is not actively used, the corresponding second-level page table can be paged out to the hard drive so it does not waste physical memory.

4 Virtual Memory

6 Multilevel Page Tables

- In a two-level page table, the virtual page number is split into two parts: the page table number and the page table offset.
- The page table number indexes the first-level page table, which must reside in physical memory.
- The first-level page table entry gives the base address of the second-level page table or indicates that it must be fetched from the hard drive when V is 0.
- The page table offset indexes the second-level page table.
- The remaining 12 bits of the virtual address are the page offset, as before, for a page size of 2^{12} =4 KB.



4 Virtual Memory

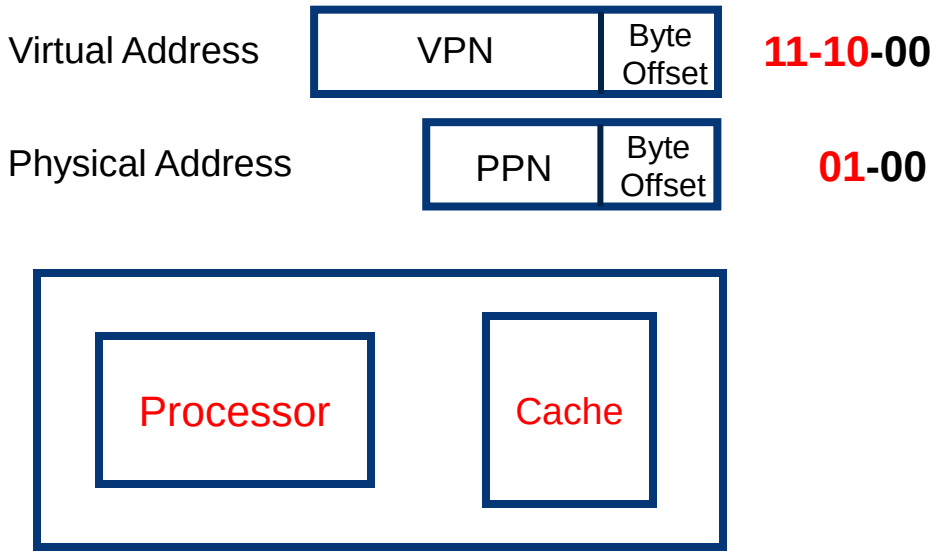
6 Multilevel Page Tables

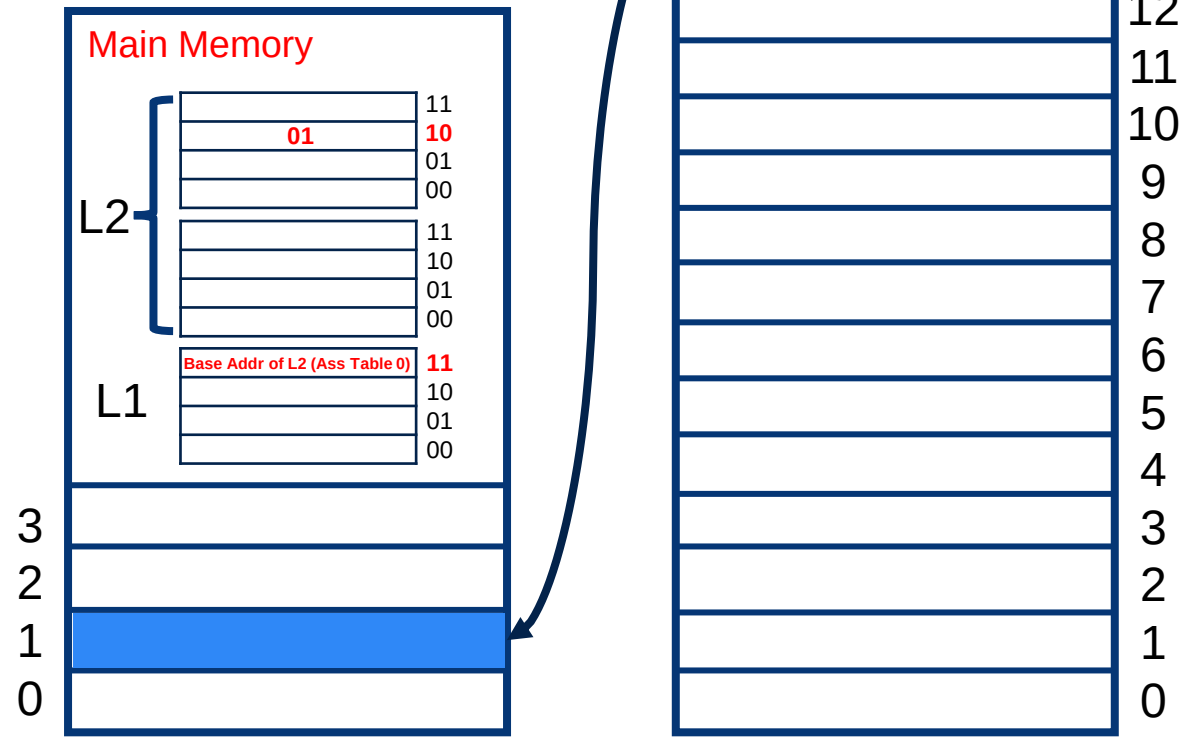
- The 19-bit virtual page number is broken into 9 and 10 bits, to indicate the page table number and the page table offset, respectively.
- Thus, the first-level page table has $2^9 = 512$ entries.
- Each of these 512 second-level page tables has $2^{10} = 1$ K entries.
- If each of the first- and second-level page table entries is 32 bits (4 bytes) and only two second-level page tables are present in physical memory at once, the hierarchical page table uses only (512 × 4 bytes) + 2 × (1 K × 4 bytes) = 10 KB of physical memory.
- The two-level page table requires a fraction of the physical memory needed to store the entire page table (2 MB).
- The drawback of a two-level page table is that it adds another memory access for translation when the TLB misses.

4 Virtual Memory

Example:

- Assume the virtual memory is divided into 16 pages, while the main memory is divided into 4 pages. So the virtual page number will be 4-bits, while the physical page number will be 4-bits.
- Assume the processor wants to access VA = 111000.

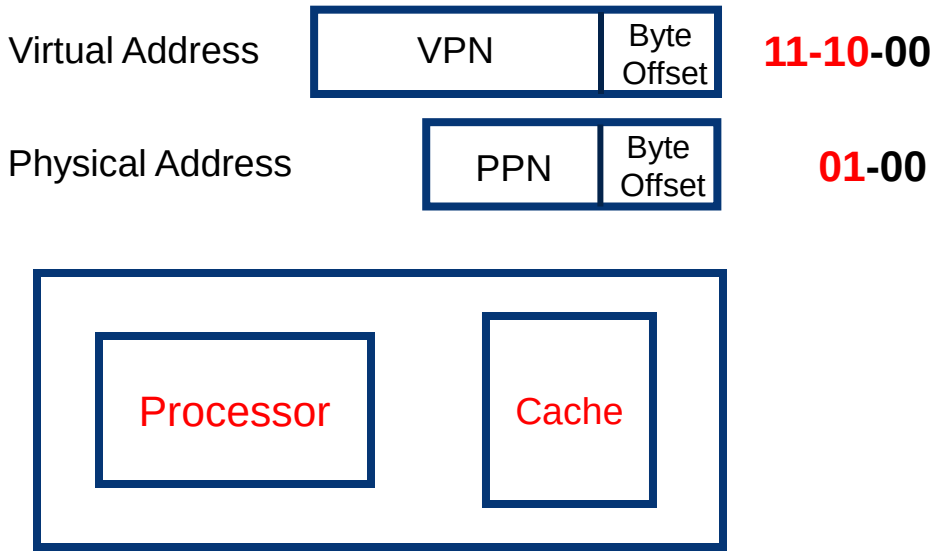


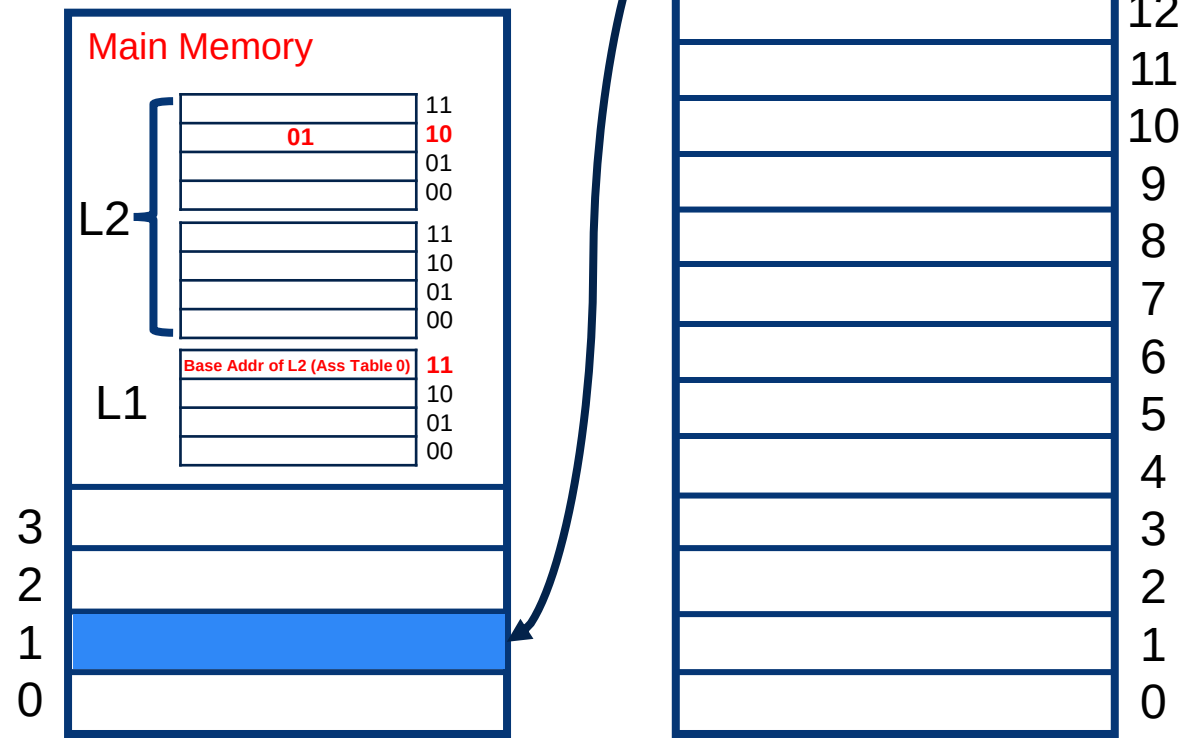


4 Virtual Memory

Example:

- If the data is not found in the cache, the processor will obtain the physical address from TLB to access the main memory.
- If it isn't found in TLB, the processor will access the L1 page table inside the main memory first to obtain the address of L2 page table to obtain the physical address.





Assignment #6

Assignment #6

- Implement a 1-level 2-way set associative 8KB cache, with 64-Byte block size.
- Do a literature survey to compare your performance with the other Pipelined MIPS Processors.
- Do a presentation to explain your work (15 mins).
- Deadline: Friday, 13th Dec 2024, at 11:59 PM.
- Deliveries: Full design, testbench, a document and a presentation for your work.